

Práctica del curso

Desarrollo dirigido por especificaciones con IA

GitHub Copilot · AGENTS.md · Skills · Tools

Abril 2026 · v2.0

OBJETIVO

Diseñar un sistema real, no solo escribir código

Diseñar un producto que pueda ser desarrollado mediante IA usando:

- **GitHub Copilot**
- **Configuración de agentes** (*AGENTS.md*)
- **Skills** (*habilidades reutilizables*)
- **Herramientas** (*MCP o APIs simuladas*)

IMPORTANTE

No se evalúa solamente la aplicación terminada.

Se evalúa la capacidad de **definir un sistema listo para ser generado por IA.**

ENUNCIADO

Diseña un producto que pueda ser desarrollado mediante IA

TRES OPCIONES DE TEMA

Elige una de estas líneas — o propón la tuya propia

1. Asistente AI para workflows

- Soporte al cliente (clasificación de tickets)
- Asistente interno de documentación
- Onboarding de desarrolladores

2. Micro-SaaS automatizado

- Extracción de datos de facturas
- Generador de contenido automático
- Conversión de inputs → acciones (emails, tareas...)

3. Aplicación reactiva / tiempo real

- Dashboard con SSE
- Sistema de alertas tipo IoT
- Eventos en streaming

RESTRICCIONES OBLIGATORIAS

Tres reglas no negociables para tu entrega

1. NO HACER “VIBE CODING”

Nada de pedirle cosas sueltas a la IA y ver qué sale

Las instrucciones del sistema deben apoyarse en un **sistema de especificaciones** y una **definición de arquitectura** explícitos, de modo que la IA tenga un marco claro sobre el que generar código.

SPEC.md

el qué

Funcionalidad, casos de uso, criterios de aceptación.

ARCHITECTURE.md

el cómo

Componentes, flujos de datos, stack y decisiones.

AGENTS.md

las reglas

Convenciones, comandos build/test, manual del repo.

SPEC.md

Fuente de verdad sobre el qué

Especificación funcional del producto. Es la primera puerta: si no está clara, la IA improvisa.

- Qué hace el producto y para quién
- Qué problema resuelve
- Casos de uso principales
- Criterios de aceptación
- Restricciones de negocio

```
# SPEC.md
```

```
## Problema
```

```
Los agentes de soporte tardan ~3 min  
por ticket en asignar categoría y  
prioridad.
```

```
## Casos de uso
```

```
1. Al recibir un ticket, el agente ve la  
categoría y prioridad sugeridas con  
nivel de confianza.
```

```
## Criterios de aceptación
```

```
- Precisión  $\geq$  85% sobre golden.jsonl  
- Latencia p95 < 800 ms  
- Soporta ES e FN
```

ARCHITECTURE.md

Fuente de verdad sobre el cómo

Definición técnica que la IA usa para construir piezas coherentes entre sí.

- Componentes del sistema
- Flujos de datos e integraciones
- Stack tecnológico
- Decisiones de diseño y trade-offs

```
# ARCHITECTURE.md
```

```
## Stack
```

- Node.js 20 + Fastify
- Claude Haiku 4.5 vía SDK
- Postgres (audit log)

```
## Flujo
```

```
Zendesk (mock) → Classifier  
  (Haiku + prompt caching) → Postgres
```

```
## Decisiones
```

- Haiku por coste/latencia
 (vs Sonnet: -3% precisión, -80% coste)
- Prompt caching sobre catálogo
 (cambia ~1 vez/mes)

AGENTS.md

Manual del proyecto para el agente de IA

Lo que el agente lee antes de tocar código. En la raíz del repo o anidado por subdirectorio.

- Cómo construir, testear y validar
- Convenciones de código y estilo
- Comandos útiles y estructura
- Reglas específicas por área
- Convive con `.github/copilot-instructions.md`

```
# AGENTS.md
```

```
## Build & test
```

- npm install && npm run build
- npm test · npm run test:golden

```
## Convenciones
```

- TypeScript estricto, sin `any`
- Endpoint nuevo → test en tests/api/
- Secretos en .env; .env.example

```
## Reglas específicas
```

- Antes de cambiar el prompt del clasificador, correr test:golden
- Cambios de schema → migración

EL CONTRATO QUE GUÍA A LA IA

Tres documentos = una IA con marco. Sin ellos, improvisa.

Si están bien definidos, **el código generado será coherente.** Si faltan, la IA improvisa.



2. COPILOT DEBE ESTAR CONFIGURADO

Mínimos obligatorios para tu entrega

1

Instrucciones claras de uso

Cómo invocar al agente, qué archivos lee, qué expectativas tiene.

2

Al menos 1 skill

Habilidad reutilizable que encapsula una tarea repetitiva del producto.

3

Al menos 1 herramienta

MCP server o mock API que reemplace dependencias externas en desarrollo.

3. ENTREGA BASADA EN SISTEMA

El resultado no es código suelto — es un repo auto-generable

LA APP

el producto funcionando

- Código fuente
- Tests automáticos
- Demo desplegable

EL SISTEMA QUE LA GENERA

documentación + configuración

- SPEC.md, ARCHITECTURE.md, AGENTS.md
- Skills y herramientas configuradas
- Copilot instructions
- **Más peso en la evaluación**

EJEMPLO PRÁCTICO

TicketTriage — un caso concreto de principio a fin

EJEMPLO: TICKETTRIAGE

Clasificador de tickets de soporte — estructura del repositorio

```
tickettriage/
├── README.md           # cómo instalar, usar, reproducir
├── SPEC.md             # qué hace el producto
├── ARCHITECTURE.md     # cómo está construido
├── AGENTS.md           # instrucciones del coding agent
├── .github/
│   ├── copilot-instructions.md
│   └── skills/
│       └── classify-ticket.md # skill reutilizable
├── tools/
│   └── mock-zendesk.ts    # mock API (o MCP server)
├── src/                # la app
└── tests/
    └── golden.jsonl      # dataset de validación
```

EJEMPLO: SKILL + HERRAMIENTA

Lo que hace que tu repo sea auto-generable por la IA

SKILL – `classify-ticket.md`

```
# classify-ticket
Dado un ticket, devuelve
{ category, priority, confidence }.

Uso: cuando el agente necesite
clasificar tickets en tests o seeds.

Pasos:
1. Cargar catálogo en categories.json
2. POST a /classify con el texto
3. Si confidence < 0.6 → uncategorized
```

TOOL – `tools/mock-zendesk.ts`

Mock de la API de Zendesk para desarrollar y testear sin credenciales reales.

Alternativa: exponerlo como **servidor MCP** consumible por Copilot u otros agentes.

Permite:

- Reproducibilidad en cualquier máquina
- Tests deterministas con datos fijos
- Onboarding sin esperar credenciales

POR QUÉ ESTO ES UN “SISTEMA”

Otra persona puede regenerar la app desde cero

“Lee SPEC.md, ARCHITECTURE.md y AGENTS.md, y genera la app.”

Y el agente sabe...

- Qué construir (*SPEC.md*)
- Cómo construirlo (*ARCHITECTURE.md*)
- Las convenciones del repo (*AGENTS.md*)
- Tareas repetitivas resueltas (*skills*)
- Acceso a dependencias externas (*tools / MCP*)

EVALUACIÓN

Qué pesa más al revisar tu trabajo

QUÉ SE EVALÚA

El código es una consecuencia de la especificación, no al revés

SPEC.md presente y claro	30 %
ARCHITECTURE.md coherente con la SPEC	20 %
AGENTS.md con build/test/convenciones	15 %
Al menos 1 skill funcional	10 %
Al menos 1 herramienta (MCP / mock)	10 %
App funcionando + tests	15 %

Pesos orientativos. Un repo bien especificado vale más que un repo con código bonito y sin SPEC.

CHECKLIST DE ENTREGA

Antes de compartir tu repo, verifica que tienes todo esto

- ☐ Tema elegido y justificado en el README
- ☐ SPEC.md con problema, casos de uso y criterios de aceptación
- ☐ ARCHITECTURE.md con stack, flujo y decisiones (con trade-offs)
- ☐ AGENTS.md con build, test, convenciones y reglas específicas
- ☐ .github/copilot-instructions.md con uso esperado
- ☐ Al menos 1 skill reutilizable en .github/skills/
- ☐ Al menos 1 herramienta (MCP server o mock API) en tools/
- ☐ App funcionando con tests pasando
- ☐ README explica cómo regenerar la app desde la IA

El entregable no es solo código.

Es un repositorio auto-generable.

GFT · Práctica del curso de Copilot

Abril 2026 · v2.0